

1. Project Background

Modern banks process millions of transactions daily across digital channels — mobile apps, ATMs, online portals, and interbank transfers. The exponential growth of financial data presents both an opportunity and a challenge: institutions that harness this data gain a decisive edge in fraud prevention, customer retention, and regulatory compliance.

FinSight is a solo capstone project that simulates a big real-world data engineering and analytics engagement for a mid-sized retail bank. The project demonstrates how a unified data platform — spanning ingestion, distributed storage, real-time processing, graph analytics, data blending, and executive reporting — can be architected and delivered using ten industry-standard technologies in a five-day solo sprint.

Client Scenario — NovaCrest Bank

NovaCrest Bank is a mid-tier retail and commercial bank headquartered in New York City, with regional operations spanning 38 states across the United States. Founded in 1987 as a community savings institution, the bank has grown through three major acquisitions over the past two decades into a full-service financial institution serving over 4.2 million individual customers and 86,000 small-to-medium business clients.

Today, NovaCrest operates 312 physical branches, 1,840 ATMs, and a digital banking platform that handles 68% of all customer interactions. The bank offers a full product suite spanning savings and current accounts, personal and car loans, mortgages, credit cards, fixed deposits, investment funds, and business banking services.

NovaCrest Bank processes over 2 million transactions per day. Leadership has identified three critical pain points:

- Inability to detect fraud in real time,
- No unified customer view across products
- Slow manual compliance reporting.

FinSight addresses all three within a single integrated platform.

2. Challenges

2.1 Data Volume & Velocity

- Millions of transactions generated daily across ATM, mobile, wire transfer, and merchant payment channels.
- Legacy batch processing introduces a 24–48 hour lag, making real-time fraud detection impossible.
- Event streams from multiple source systems arrive in different formats and at varying rates.

2.2 Data Variety

- Structured data: transaction records, account details, and balance sheets in relational/tabular formats.
- Semi-structured data: customer KYC profiles and preference objects stored as JSON documents.
- Graph data: account-to-account transaction networks that reveal fraud rings and money laundering patterns invisible to tabular analysis.

2.3 Analytical Complexity

- Fraud detection requires combining historical patterns (batch) with live transaction scoring (streaming).
- Customer 360 analysis requires joining data from multiple systems that were never designed to interoperate.
- Compliance reporting demands auditability, reproducibility, and delivery on tight regulatory deadlines.

2.4 Tooling Gaps

- No single existing tool handles all three workloads — streaming analytics, batch computation, and graph traversal.

- Business users cannot access raw data — a transformation and self-service layer is essential before insights can be consumed.

3. Business Need

NovaCrest Bank requires a scalable, integrated data platform that delivers four measurable business outcomes:

#	Business Need	Success Metric
01	Real-time fraud detection on incoming transactions	Fraudulent transactions flagged within 2 seconds of stream arrival
02	Unified customer analytics across all product lines	Single customer view dashboard: transactions, profile, and composite risk score
03	Automated regulatory and risk compliance reports	Weekly compliance report generated without any manual analyst intervention
04	Graph-based identification of fraud rings and money laundering networks	Suspicious account clusters surfaced via Neo4j graph traversal queries

4. Attributes & Sample Data

4.1 Primary Dataset : Transactions.csv

6.3 million synthetic mobile money transactions simulating real financial behaviour, including labelled fraud records.

Attribute	Description	Type	Sample Value
step	Time step (1 step = 1 hour)	Integer	1
type	Transaction category	String	TRANSFER, PAYMENT, CASH_OUT
amount	Transaction amount (USD)	Float	9,839.64
nameOrig	Originating account ID	String	C1231006815
oldbalanceOrg	Sender balance before	Float	170,136.00
newbalanceOrig	Sender balance after	Float	160,296.36
nameDest	Destination account ID	String	M1979787155
oldbalanceDest	Recipient balance before	Float	0.00
newbalanceDest	Recipient balance after	Float	0.00
isFraud	Ground truth fraud label	Integer	0 = normal 1 = fraud
isFlaggedFraud	System-flagged large transfer	Integer	0 = normal 1 = flagged

4.2 Customer Profile Dataset (MongoDB) : novacrest_customers

A synthetic JSON dataset of 10,000 customer records loaded into MongoDB. The customerId field links back to nameOrig / nameDest columns in PaySim, enabling joins in Alteryx.

Sample MongoDB Customer Document

{

```

"customerId"      : "C1231006815",
"name"            : "Aisha Patel",
"age"             : 34,
"segment"         : "Premium",
"products"        : ["savings", "mortgage", "credit_card"],
"kyc_status"      : "verified",
"risk_score"      : 0.12,
"churn_probability" : 0.05,
"account_opened"  : "2019-03-14",
"preferred_channel" : "mobile"
}

```

4.3 Neo4j Graph Schema

The fraud knowledge graph uses two node types and one relationship type to model transaction networks across customer accounts.

Graph Element	Type	Properties
Account	Node	accountId, balance, accountType (CASH / MERCHANT)
Transaction	Node	txnId, amount, step, isFraud
SENT	Relationship	amount, timestamp, transactionType
RECEIVED_BY	Relationship	newbalanceDest, isFraud

NODE neo4j_accounts_nodes.csv · 499 records :Account		
accountId:ID	accountType	:LABEL
C3746317213	CUSTOMER	Account
C1231006815	CUSTOMER	Account
C8831028753	CUSTOMER	Account
M4755334423	MERCHANT	Account
M9271938450	MERCHANT	Account

NODE neo4j_transaction_nodes.csv · 1,554 records :Transaction					
txnId:ID	step:int	type	amount:float	isFraud:int	:LABEL
TXN0000001	1	TRANSFER	229133.24	1	Transaction
TXN0000002	1	CASH_OUT	181935.41	1	Transaction
TXN0000003	2	TRANSFER	45200.00	0	Transaction
TXN0000004	2	CASH_OUT	88750.12	0	Transaction
TXN0000005	3	TRANSFER	312400.00	1	Transaction

Full graph traversal path:

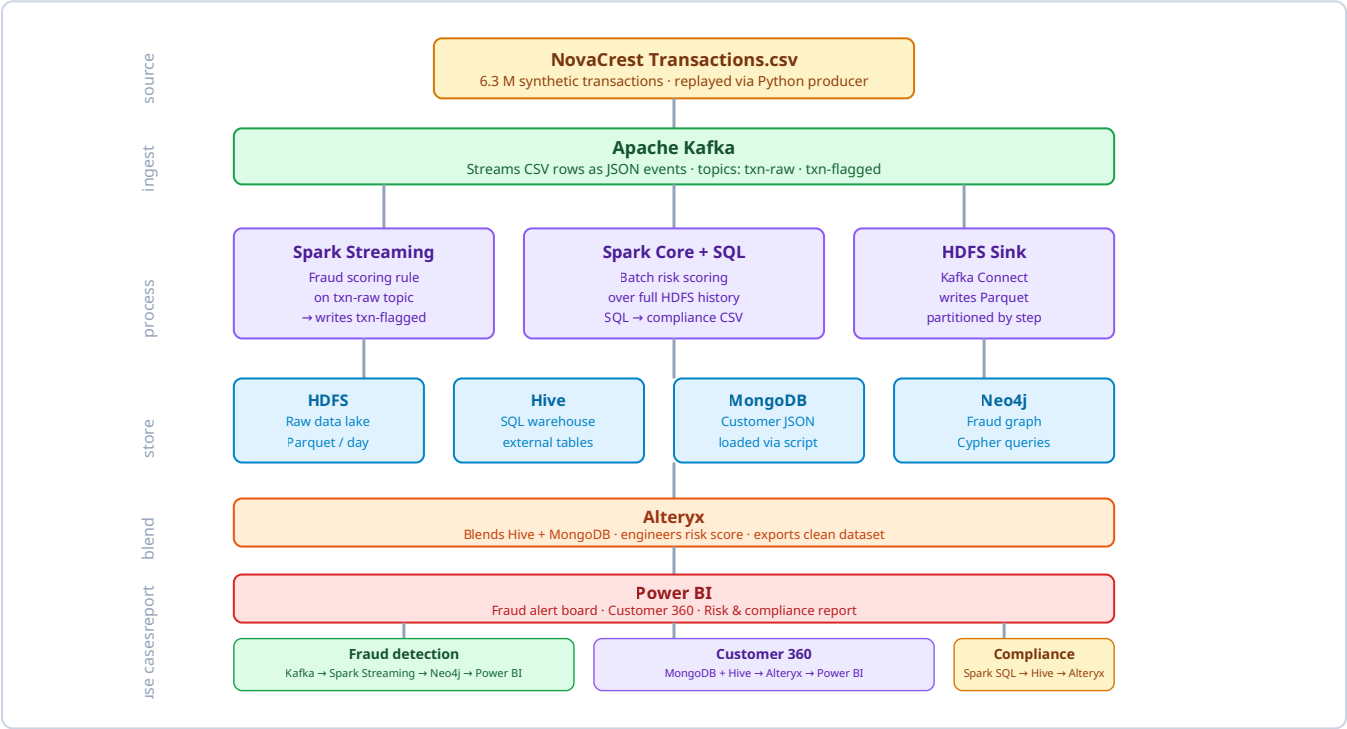
(Account) -[:SENT]-> (Transaction) -[:RECEIVED_BY]-> (Account)

499 account nodes · 1,554 transaction nodes · 3,108 relationship edges total

5. Proposed Solution & Solution Architecture

5.1 Solution Overview

FinSight is a big five-layer data platform. Each layer is served by one or more of the ten required technologies. Data flows from raw event ingestion through distributed storage, real-time and batch processing, multi-model databases, and self-service analytics — terminating in executive-grade Power BI dashboards.



Layer	Technology	Role in FinSight
Ingestion	Kafka	Streams PaySim transactions into two topics: txn-raw (all events) and txn-flagged (fraud output)
Raw Storage	HDFS	Data lake — all Kafka events landed here first in Parquet format, partitioned by day
Warehouse	Hive	External SQL tables over HDFS — exposes the data lake for Spark SQL and Alteryx queries
Document Store	MongoDB	Customer KYC profiles and product preferences stored as JSON documents
Graph Database	Neo4j	Account-transaction graph enabling Cypher-based fraud ring and money mule detection
Stream Processing	Spark Streaming	Micro-batch fraud scoring on live Kafka stream — flags suspicious transactions in near real-time
Batch Processing	Spark Core	Overnight batch risk calculations, customer segmentation, and rolling 7-day trend analysis
SQL Analytics	Spark SQL	Ad-hoc SQL queries over Hive tables — generates weekly compliance summary tables
Data Blending	Alteryx	Joins Hive + MongoDB outputs, engineers composite risk and churn features, exports for Power BI
Visualisation	Power BI	Three interactive dashboards: Fraud Alert Board, Customer 360, Risk & Compliance Report

5.2 Pipeline Flow

- The NovaCrest Transactions CSV must be replayed into Apache Kafka via a Python producer script, alongside the generated customer JSON dataset.
- Kafka must route all transaction events to the txn-raw topic; the Spark Streaming fraud scoring job must write flagged records to the txn-flagged topic.
- A Kafka Connect HDFS Sink Connector must be configured to land all txn-raw events to HDFS in Parquet format, partitioned by transaction step.
- External Hive tables must be created over the HDFS Parquet files to expose the data lake for SQL querying without duplicating the underlying data.
- Spark Streaming must consume the txn-raw topic, apply the fraud scoring rule, and persist flagged results back to HDFS.
- A Spark Core nightly batch job must compute rolling 7-day risk scores per customer account from the full HDFS transaction history.
- Spark SQL must query the Hive warehouse to produce a weekly compliance summary, which must then be consumed downstream by the Alteryx blending workflow.
- MongoDB must be loaded with the customer profile JSON dataset using mongoimport before the Alteryx workflow is executed.
- Neo4j must be populated with account nodes, transaction nodes, and relationship edges; the fraud ring Cypher query must then be run to validate the graph.
- Alteryx must blend the Hive compliance output with MongoDB customer profiles and engineer the composite risk score and churn features required by Power BI.

- Power BI must connect to the Alteryx output datasets to deliver three audience-specific interactive dashboards as the final project deliverable.

6. Data Ingestion

6.1 Kafka Topic Setup

Two Kafka topics must be created at the start of Day 1. The **txn-raw** topic requires three partitions to support parallel consumer groups — one for Spark Streaming and one for the HDFS Sink Connector. The **txn-flagged** topic requires a single partition, as fraud output volume is expected to be low.

6.2 Python Kafka Producer

A Python producer script must be implemented to read the **NovaCrest Transactions CSV** row by row and publish each transaction as a serialized JSON message to the **txn-raw** topic. The producer should target approximately 1,000 messages per second to simulate a realistic transaction stream without overwhelming a local Kafka setup.

6.3 HDFS Landing Structure

A Kafka Connect **HDFS Sink Connector** must be configured to subscribe to the **txn-raw** topic and write incoming events to HDFS as **Parquet files**, partitioned by the step field. The directory structure defined below must be established before Day 2 processing begins, as all subsequent Spark and Hive operations depend on reading from these paths.

7. Spark Processing

7.1 Spark Streaming — Real-Time Fraud Detection

A PySpark Structured Streaming job must be implemented to consume the **txn-raw** Kafka topic in micro-batches and apply a real-time fraud scoring rule. A transaction must be flagged when all three of the following conditions are met: the transaction type is TRANSFER or CASH_OUT, the amount exceeds \$200,000, and the destination post-transaction balance is zero — the classic account-emptying pattern. Flagged records must be written to the **txn-flagged** topic.

Additional Requirements — Spark Streaming

R1: The streaming job must implement a checkpoint directory at `/finsight/checkpoints/fraud` to guarantee exactly-once processing semantics and enable automatic recovery if the job is interrupted mid-stream.

R2: The job must also compute and log a running fraud rate metric per micro-batch — expressed as $(\text{flagged count} / \text{total count} \times 100)$ — and write this summary to `/finsight/processed/streaming_metrics/` on HDFS for monitoring purposes.

7.2 Spark Streaming — Real-Time Customer Churn Detection

A second independent PySpark Structured Streaming application must be implemented to detect early churn signals in real time from the **txn-raw** Kafka topic. Unlike the fraud detection job which operates at the transaction level, this application must operate at the customer level — maintaining a rolling behavioural profile per customer across a sliding time window and raising a churn alert when that profile crosses predefined thresholds.

A customer must be flagged as a churn risk when any two or more of the following signals are observed within a 24-step (24-hour) sliding window:

- Transaction frequency drops below 1 transaction per 12 steps for a customer with a historical average above 3 per 12 steps.
- The customer's average transaction amount in the window falls below 20% of their all-time average amount.

- The customer's transaction type shifts exclusively to CASH_OUT with no PAYMENT or DEBIT activity — indicating account liquidation behaviour.
- The post-transaction balance (newbalanceOrig) reaches zero or drops below \$500 for two or more consecutive transactions.

Churn-flagged customer records must be written to a dedicated Kafka topic (txn-churn) and simultaneously persisted to HDFS at /finsight/processed/churn_alerts/ in Parquet format for downstream consumption by the Alteryx Customer Risk Blend workflow and the Power BI Customer 360 dashboard.

Additional Requirements — Churn Detection

R1: The churn detection job must run concurrently alongside the fraud detection job — both streaming applications must be active simultaneously during the Day 5 demo to demonstrate that NovaCrest's platform can support multiple independent real-time workloads on the same Kafka topic without interference.

R2: A third Kafka topic (txn-churn) must be created with a single partition before this job is started. The churn alert records written to this topic must include the customerId, the window start and end timestamps, and the specific signal(s) that triggered the flag — enabling the Power BI Customer 360 dashboard to display a churn risk indicator alongside the customer's transaction history.

7.3 Spark Core — Batch Risk Scoring

A Spark Core batch job must be developed to run nightly over the full HDFS transaction history. The job must compute a rolling 7-day composite risk score per customer account, incorporating four weighted factors: transaction frequency, average transfer amount, CASH_OUT proportion, and the count of unique destination accounts reached.

- **Input:** The job must read from /finsight/raw/transactions/ across all Parquet partitions.
- **Output:** Results must be written to /finsight/processed/risk_scores/ — one row per customerId with a normalised risk_score between 0 and 1.
- **Schedule:** The job must be schedulable via cron or Apache Airflow to trigger automatically after market close each day.

Additional Requirements — Spark Core

R1: The batch job must apply a customer segmentation step that classifies each customer into one of three risk tiers — Low (risk_score < 0.25), Medium (0.25–0.60), and High (> 0.60) — and include this tier as an additional column in the output dataset for use in Power BI filtering.

R2: The job must produce a second output file at /finsight/processed/daily_summary/ containing transaction volume, total amount, and fraud count aggregated by transaction type and step — this file is required as input to the Alteryx Transaction Summary workflow on Day 4.

7.4 Spark Core — Customer Lifetime Value Scoring

A second Spark Core batch application must be implemented to compute a Customer Lifetime Value (CLV) score for every account in the transaction dataset. While the risk scoring job (Section 7.3) focuses on threat indicators, the CLV job must focus on revenue potential — quantifying each customer's historical contribution to the bank and projecting their future value to support targeted retention and cross-sell decisions in the Power BI Customer 360 dashboard.

The CLV score must be computed using the following four weighted components, each derived entirely from the NovaCrest Transactions dataset:

- **Transaction Volume Score (30%):** Total cumulative transaction amount for the customer across all steps, normalised to a 0–1 scale relative to the highest-spending account in the dataset.
- **Transaction Frequency Score (25%):** Total number of transactions per customer, normalised to a 0–1 scale relative to the most active account.

- **Product Diversity Score (25%):** Count of distinct transaction types used by the customer (PAYMENT, TRANSFER, CASH_IN, DEBIT, CASH_OUT), divided by 5 — a proxy for product breadth in the absence of a full product holdings table.
- **Recency Score (20%):** Inverse of the number of steps since the customer's last transaction, normalised so that more recent activity scores higher. Customers with no activity in the last 48 steps must receive a recency score of 0.

The final CLV score must be the weighted sum of all four components, expressed as a value between 0 and 1. Customers scoring above 0.70 must be classified as High Value, 0.40–0.70 as Growth Potential, and below 0.40 as At Risk. The output must be persisted to HDFS at `/finsight/processed/clv_scores/` and must be joinable to the MongoDB customer collection via `customerId` for use in the Alteryx Customer Risk Blend workflow.

Additional Requirements — CLV Job

R1: The CLV output at `/finsight/processed/clv_scores/` must be registered as a second external Hive table (`finsight.customer_clv`) so that it is queryable alongside the transactions table via Spark SQL without requiring a separate file read in each downstream job.

R2: The CLV job must be designed to run independently of the risk scoring job and must not share any intermediate state or output paths with it. Both jobs must be executable on the same Spark cluster without resource contention — each must define its own `SparkSession` with a distinct `appName` to enable separate monitoring in the Spark UI during the Day 5 demo.

7.5 Spark SQL — Compliance Aggregation

A Spark SQL query must be written against the Hive warehouse to produce the weekly compliance summary required by the Alteryx blending workflow and the Power BI Risk & Compliance dashboard. The query must aggregate transaction volume, fraud counts, and fraud rate by transaction type across a 168-step (7-day) window.

Additional Requirements — Spark SQL

R1: A second Spark SQL query must be written to produce a customer-level fraud summary — aggregating total transaction count, total amount, confirmed fraud count, and fraud rate per `nameOrig` — and the output must be saved to `/finsight/processed/customer_fraud_summary/` for use as Input A in the Alteryx Customer Risk Blend workflow.

R2: Both Spark SQL queries must be wrapped in a single executable Python script (`spark_sql_jobs.py`) that accepts a `--mode` argument to run either the compliance query or the customer summary query independently, enabling them to be triggered separately from the Day 5 demo without restarting the full Spark session.

7.6 Spark SQL — Account Dormancy Report

A second Spark SQL application must be implemented to produce the Account Dormancy Report — a regulatory-grade output that identifies customer accounts that have shown no transaction activity within a defined inactivity window. Account dormancy reporting is a compliance obligation under NovaCrest's internal policy and must be produced on a weekly basis alongside the compliance aggregation query.

An account must be classified as dormant if it meets all of the following criteria as evaluated against the full transaction history in the Hive warehouse:

- The account's most recent transaction step (`last_active_step`) falls more than 72 steps (72 hours) before the maximum step value in the dataset — indicating 3 or more days of complete inactivity.
- The account has a recorded transaction history of at least 5 prior transactions — excluding newly opened accounts with insufficient history from the dormancy classification.
- The account type is CUSTOMER (`nameOrig` starting with 'C') — merchant accounts (prefix 'M') must be excluded from dormancy classification as they operate on different activity cycles.

The dormancy report output must be written to `/finsight/processed/dormancy_report/` as a Parquet file and must also be saved as a CSV to `/finsight/exports/dormancy_report.csv` for direct consumption by the Alteryx Customer Risk Blend workflow. The report must be joinable to the MongoDB customer collection via the `customerId` field.

Additional Requirements — Dormancy Report

R1: The dormancy report must apply two severity classifications — Dormant (72–120 steps of inactivity) and Severely Dormant (more than 120 steps of inactivity) — so that the Power BI Compliance dashboard can display a tiered dormancy breakdown chart that distinguishes recently inactive accounts from long-term dormant ones requiring escalation.

R2: The dormancy report query must be added as a third mode (`--mode dormancy`) to the `spark_sql_jobs.py` script defined in Section 7.5, so that all three SQL jobs — compliance aggregation, customer fraud summary, and account dormancy — can be triggered from a single script entry point during the Day 5 demo walkthrough.

8. Database Configuration

8.1 Hive — Data Warehouse Layer

An external Hive table must be created over the HDFS Parquet files using the DDL specification below. External tables are required — not managed tables — to ensure that the underlying HDFS data is never deleted if the table is dropped, and to allow Spark, Hive, and Alteryx to read concurrently from the same files without data duplication.

Additional Requirements — Hive

R1: A second Hive view must be created on top of the `finsight.transactions` table named `finsight.vw_fraud_transactions`, filtering to only rows where `isFraud = 1`. This view must be used as the source for the Neo4j graph load and the Power BI Fraud Alert Board, ensuring a single consistent definition of a confirmed fraud record is shared across all downstream consumers.

R2: Table statistics must be computed on `finsight.transactions` using `ANALYZE TABLE finsight.transactions COMPUTE STATISTICS` to enable the Hive query optimiser to generate efficient execution plans. This step is required before any Spark SQL queries are run against the warehouse.

8.2 Hive — Transaction Summary Mart

A second Hive table must be created to serve as a pre-aggregated transaction summary mart — a curated, query-optimised layer built on top of the raw `finsight.transactions` table. This mart must consolidate daily transaction statistics per customer into a single queryable table that significantly reduces the computational cost of repeated aggregation queries run by Spark SQL, Alteryx, and Power BI during the Day 5 demo.

Unlike the raw external table which stores one row per transaction, the summary mart must store one row per customer per step — pre-computing the metrics most frequently required by downstream consumers. The mart must be created as a managed internal Hive table populated by an `INSERT OVERWRITE` statement from the raw transactions table, and must be refreshed each time the Spark Core batch job completes.

The transaction summary mart must include the following columns, all derived from `finsight.transactions`:

- **customerId:** The originating account identifier (`nameOrig`), serving as the primary join key to MongoDB and the CLV output.
- **step:** The time step of the aggregated window — one row per customer per step.
- **txn_count:** Total number of transactions initiated by the customer in that step.
- **total_amount:** Sum of all transaction amounts for the customer in that step.

- **avg_amount:** Average transaction amount for the customer in that step.
- **max_amount:** Maximum single transaction amount in that step — used to detect outlier transactions in Power BI.
- **fraud_count:** Count of transactions where `isFraud = 1` for the customer in that step.
- **txn_types:** Comma-separated list of distinct transaction types used by the customer in that step.
- **last_balance:** The `newbalanceOrig` value from the customer's most recent transaction in that step — a proxy for end-of-step balance.

Additional Requirements — Transaction Summary Mart

R1: The summary mart must be registered with Hive statistics immediately after population by running `ANALYZE TABLE finsight.txn_summary_mart COMPUTE STATISTICS FOR COLUMNS` so that all downstream Spark SQL queries joining against the mart benefit from column-level statistics in the query optimiser — this is especially important for the compliance aggregation and dormancy report queries which perform range scans on the step column.

R2: The Alteryx Transaction Summary workflow (Section 9.2) must be updated to read from `finsight.txn_summary_mart` via the Hive ODBC connection rather than from the raw CSV export produced by Spark Core. This ensures that the Alteryx output reflects the same pre-aggregated definitions used by the Power BI dashboards, eliminating any metric discrepancy between the two reporting surfaces.

8.3 MongoDB — Customer Document Store

The 10,000 customer JSON documents provided in `noveacrest_customers.json` must be imported into MongoDB using the `mongoimport` command below. Each document must be keyed on the `customerId` field, which is designed to match the `nameOrig` and `nameDest` columns in the transaction dataset — this join key must be preserved without modification to enable correct blending in Alteryx.

Additional Requirements — MongoDB

R1: After import, a compound index must be created on the `customers` collection covering `customerId` and `segment` fields. This index is required to ensure that Alteryx join operations and Power BI Customer 360 queries perform acceptably against the 10,000-document collection without full-collection scans.

R2: A MongoDB aggregation pipeline query must be written and executed to validate the import — grouping documents by segment and returning the count per segment (Premium, Standard, Basic, Private Banking, Student). The output must match the expected distribution defined in the sample data specification before the Day 4 Alteryx workflow is run.

8.4 Neo4j — Fraud Knowledge Graph

The Neo4j graph must be populated using the provided Python loader script (`neo4j_loader.py`) and the four CSV source files. The graph must implement a two-relationship model — `(Account)-[:SENT]->(Transaction)-[:RECEIVED_BY]->(Account)` — so that each Transaction is a first-class node. This structure is required because transaction properties such as `amount`, `isFraud`, and `step` must be queryable directly on the graph path. Once loaded, the Cypher fraud ring query below must be executed to identify accounts with more than three distinct inbound senders — the primary money mule detection signal.

9. Alteryx — Data Blending & Feature Engineering

Two Alteryx Designer workflows must be built on Day 4 using the visual drag-and-drop canvas. Each workflow must blend, clean, and engineer features from upstream data sources without requiring custom code. The output datasets from both workflows must be saved in formats compatible with Power BI import.

9.1 Workflow 1 — Customer Risk Blend

- **Input A:** The Spark SQL compliance summary CSV exported from Hive — containing transaction volumes and fraud counts per customer — must be connected as the first input.
- **Input B:** The MongoDB customer export JSON — containing segment, product holdings, KYC status, and churn probability — must be connected as the second input.
- **Formula Tool:** A composite risk score must be computed using the formula: $(\text{fraud_rate_pct} \times 0.6) + (\text{churn_probability} \times 0.4)$.
- **Output:** The blended dataset must be exported as a clean XLSX file for import into the Power BI Customer 360 and Compliance dashboard pages.

9.2 Workflow 2 — Transaction Summary

- **Input:** The CSV export from the Spark Core nightly batch job — containing HDFS daily transaction aggregates — must be connected as the input source.
- **Filter:** A filter must be applied to retain only records where step values fall between 1 and 168, covering the first 7 simulation days.
- **Aggregate:** Compute total volume, average transaction amount, and fraud count grouped by transaction type and step.
- **Output:** The summarised output must be exported as a CSV file for import into the Power BI Fraud Alert Board dashboard page.

10. Power BI Dashboards

Three report pages are delivered in a single .pbix file. Each page is designed for a distinct business audience with visuals tailored to their decision-making needs.

Page	Dashboard Name	Primary Audience	Key Visuals
1	Fraud Alert Board	Fraud / Risk Team	Live flagged transaction table, fraud rate KPI card, volume by type bar chart, daily fraud trend line
2	Customer 360	Relationship Managers	Risk score scatter plot, segment donut chart, churn heatmap, product holdings matrix
3	Risk & Compliance Report	Compliance Officers	Weekly fraud rate trend, total volume by type, top 20 flagged accounts, compliance KPI summary cards

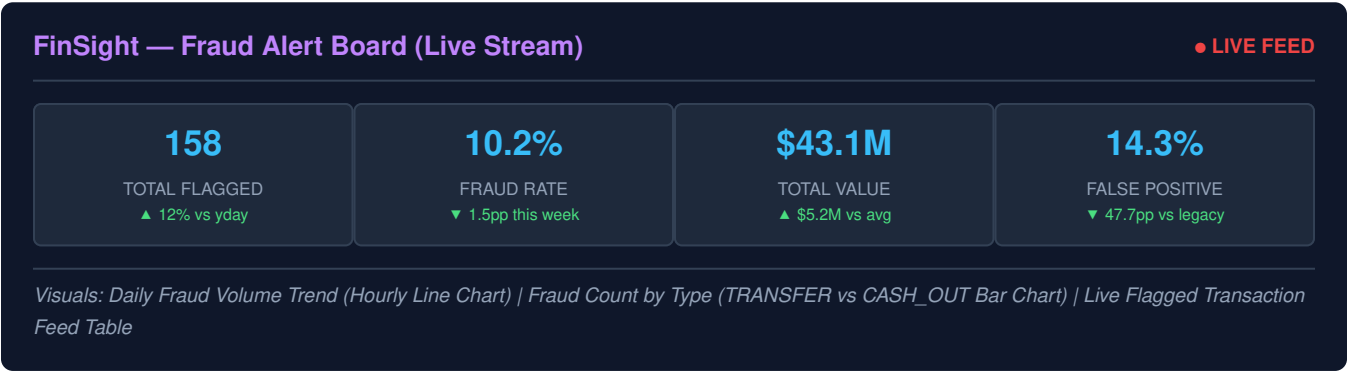
Page 1 — Fraud Alert Board

Audience: Fraud & Risk Team · Source: txn-flagged Kafka topic + Alteryx output

Overview: The Fraud Alert Board must provide the fraud and risk team with a live, continuously refreshed view of all transactions flagged by the Spark Streaming fraud scoring job. The dashboard must surface four key performance indicators at a glance, a time-series trend of fraud volume by transaction type, a breakdown of fraud counts across transaction categories, and a scrollable table of the most recently flagged transactions with their confirmation status.

- **KPI 1:** Total fraud transactions flagged in the current day — with day-on-day delta.
- **KPI 2:** Fraud rate as a percentage of all transactions — with week-on-week change.
- **KPI 3:** Total monetary value of flagged transactions — with comparison to daily average.
- **KPI 4:** False positive rate — must show improvement vs the legacy 62% baseline.

Sample Outcomes & Layout:



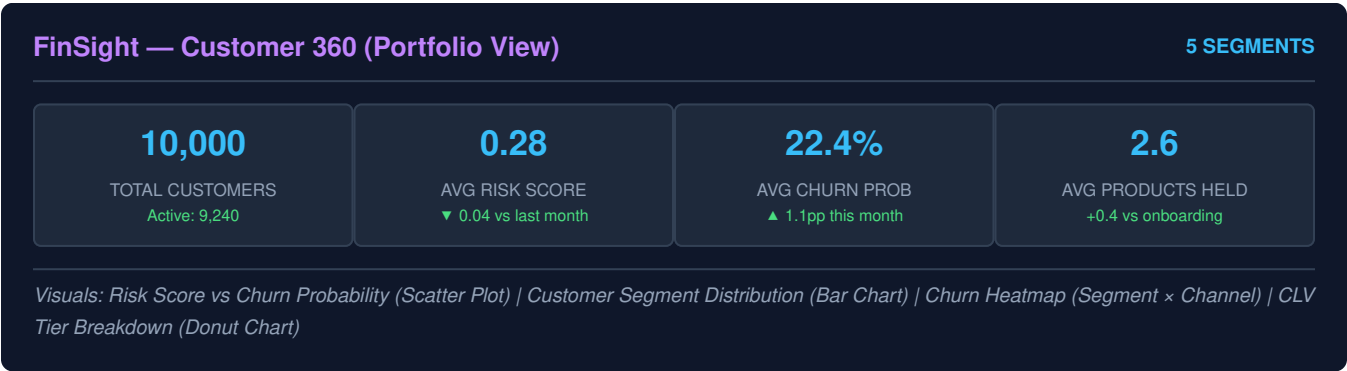
Page 2 — Customer 360

Audience: Relationship Managers · Source: Alteryx Customer Risk Blend

Overview: The Customer 360 dashboard must give relationship managers a unified view of each customer’s behavioural profile, risk exposure, and churn likelihood — data that was previously spread across four disconnected systems at NovaCrest. All metrics on this page are derived from the Alteryx Customer Risk Blend workflow output, which joins the Hive compliance table with the MongoDB customer profiles and appends the CLV score computed by the Spark Core CLV job.

- **Scatter plot:** Risk score vs churn probability coloured by customer segment — allows relationship managers to identify high-risk, high-churn clusters requiring immediate outreach.
- **Segment bars:** Customer count by segment (Standard, Basic, Premium, Student, Private Banking) — shows portfolio composition at a glance.
- **Churn heatmap:** Churn probability per segment per preferred channel — identifies which channel combinations carry the highest attrition risk.
- **CLV donut:** Customer lifetime value tier distribution (High Value / Growth Potential / At Risk) — guides prioritisation of retention effort.

Sample Outcomes & Layout:



Page 3 — Risk & Compliance Report

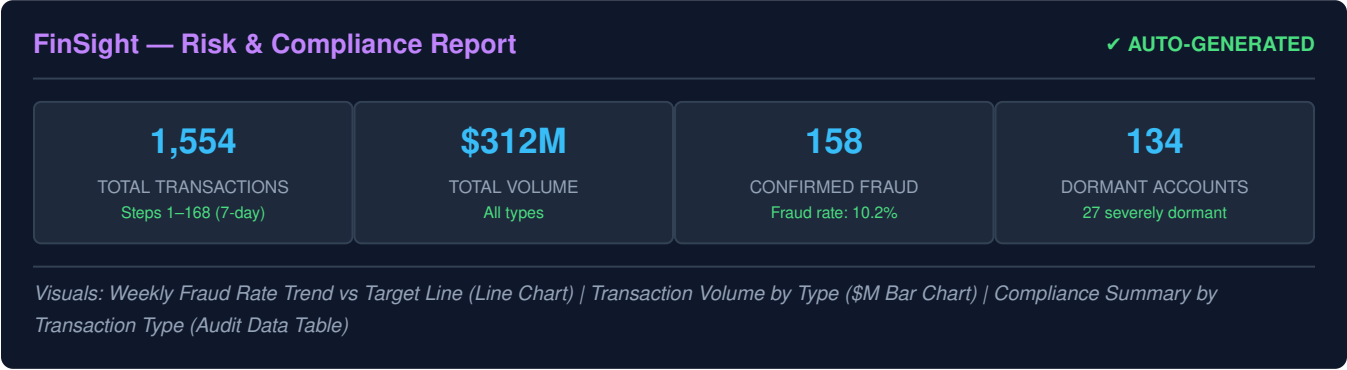
Audience: Compliance Officers · Source: Spark SQL → Hive → Alteryx Transaction Summary

Overview: The Risk & Compliance Report must replace NovaCrest’s existing 18-working-day manual quarterly process with an automated weekly dashboard that is always current. All data is sourced from the Spark SQL compliance aggregation query output, joined with the Hive transaction summary mart and enriched by the Alteryx Transaction Summary workflow. The page must include a transaction type filter to allow compliance officers to drill into specific categories during regulatory review.

- **Fraud rate trend:** Weekly fraud rate (%) plotted as a line chart against a target threshold line — enables the compliance team to monitor trajectory and detect deterioration early.

- **Volume by type:** Total transaction volume in USD by type (TRANSFER, CASH_OUT, PAYMENT, CASH_IN, DEBIT) — required for BSA large currency transaction reporting.
- **Compliance table:** Full summary table with transaction type, count, volume, fraud count, fraud rate, and risk classification — this table is the primary audit artefact submitted to regulators.
- **Dormancy KPI:** Count of dormant and severely dormant accounts — feeds the account dormancy report requirement (Section 7.6).

Sample Outcomes & Layout:



11. 5-Day Execution Plan

Day	Tasks	Deliverable	Tools Used
Day 1	Set up Kafka topics, implement the Python producer, configure the HDFS Sink Connector, and verify Parquet landing	A working Kafka-to-HDFS pipeline must be demonstrated with date-partitioned Parquet files confirmed on HDFS	Kafka, HDFS
Day 2	Write Spark Streaming fraud job, Spark Core batch risk scoring, Spark SQL compliance query	3 PySpark scripts; flagged dataset; compliance summary CSV on HDFS	Spark Streaming, Spark Core, Spark SQL
Day 3	Create external Hive tables, import MongoDB customer data, load the Neo4j graph using the loader script, and execute the Cypher fraud ring query	All 3 databases must be populated and queryable; the Cypher fraud ring query must return at least 5 suspicious accounts	Hive, MongoDB, Neo4j
Day 4	Build 2 Alteryx workflows: the customer risk blend and the transaction daily summary	2 Alteryx .yxmd workflow files must be saved and clean XLSX and CSV output files must be verified and ready for Power BI import	Alteryx
Day 5	Build 3 Power BI dashboard pages, connect to the Alteryx output datasets, and prepare the end-to-end demo walkthrough	A .pbix file containing 3 interactive dashboard pages must be completed and a structured demo script covering the full pipeline must be prepared	Power BI

Solo Completion Strategy

Scope discipline is key to five-day solo delivery. Each technology is used for exactly one focused task. Fallback rule: if Day 1 Kafka-to-HDFS setup runs over time, switch to direct file load to HDFS and revisit Kafka as a Day 5 stretch goal. The core pipeline — HDFS → Spark → Hive → Alteryx → Power BI — must be fully functional before any advanced features are attempted.

12. Expected Outcomes

- A fully functional end-to-end big data pipeline demonstrating all 10 required technologies in a single coherent architecture.
- Real-time fraud detection with sub-second latency on the Kafka → Spark Streaming path, validated against PaySim ground truth labels.
- A Neo4j graph exposing at least 5 high-risk account clusters identified from fraud records in the PaySim dataset.
- Three Power BI dashboards delivering actionable insights to fraud risk, relationship management, and compliance teams.
- Complete project documentation covering architecture, pipeline logic, schemas, code snippets, and a demo walkthrough guide.

10 Technologies integrated	5 Days to full delivery	3 Power BI dashboards	6.3M Transaction records
--	---	---	--